

Take-Home Assignment — AI Product Engineer Intern

Time budget: 7 calendar days from the day you receive this. We expect 15–20 hours of actual work. If you find yourself at 40 hours, you have misjudged the scope — stop, ship what you have, and write down what you cut.

Read in this order: this brief → 01-problem-context.md → 02-data-and-systems.md → 03-deliverables-and-submission.md → 04-evaluation.md → 05-faq.md.

The situation

You have been hired by the **Karnataka State Power Distribution Board** (fictional, but modelled closely on how real ESCOMs in Karnataka operate) as their first product engineer.

Here is their problem, in their words:

When a domestic supply line develops a fault — a snapped low-tension wire, a blown fuse at a distribution transformer, a cut jumper — the electricity goes out for a cluster of houses. Our control room finds out when people start calling the complaint number.

*From that first call, it takes us **at least two hours** to work out which specific span of wire has failed. A lineman drives to the area and works backwards from the dark houses, pole by pole, until he finds the break. Only then do we know what vehicle, what material, and what crew to send.*

Outage frequency has gone up. We are rewiring the city, but that is a multi-year programme. In the meantime we need to cut that two-hour identification window down to minutes.

What they want from the software: the moment a fault happens, the control room should know *where* it is — the exact span of line, coordinates precise enough to drive to, and the PIN code. Dispatching the crew is their job, not yours. But once the crew fixes it, the system must confirm from the field data that power is actually flowing again, and close the ticket.

What you have to work with

The department has already put an IoT device on most of their distribution poles. Each device reports **one thing**: whether that pole is energized or not. There is a lamp point at the pole, and the device knows whether it is live.

You also have a registry of every pole with its GPS coordinates.

That's it. There is **no sensor on the wire itself**. You cannot measure current, direction of flow, or impedance. You know pole-by-pole liveness and pole-by-pole location, and you have to get from that to "the fault is on the span between pole P-2211 and pole P-2212, at 12.9682° N 77.5946° E, PIN 560078."

`02-data-and-systems.md` gives you the exact payload formats, the scale, and the ways this data is dirty. Read it carefully — some of the constraints in there are the actual difficulty of the problem, not decoration.

What you have to build

A working system, deployed and reachable on a public URL, that does all of this:

1. Ingest

Accept telemetry from the pole devices. Your design should be honest about volume, ordering, duplication, and device failure.

2. Detect and localize

Turn a stream of "pole is dark" signals into a small number of **located faults**. For each one, output at minimum:

- the specific span or asset you believe has failed
- coordinates you would put into a vehicle's navigation
- the PIN code
- how many poles are affected downstream
- how confident you are, and why

A control room that receives 40 separate alerts for one snapped wire is worse than no system at all. Grouping is part of the problem.

3. Don't cry wolf

Some poles go dark for reasons that are not faults. Some devices die while the power is fine. There is scheduled load shedding. Your system has to be trustworthy enough that an operator doesn't start ignoring it in week two.

4. Ticket workflow

Every detected fault becomes a ticket that moves through a lifecycle: detected → acknowledged → crew assigned → resolved → verified → closed.

Restoration must be verified from telemetry, not from someone clicking a button. When the affected poles come back to life, the system should say so on its own. If a lineman marks it fixed and the poles are still dark, the system should not believe him.

5. An operator console

A UI for the person sitting in the control room at 2 a.m. They are not an engineer. They need to see, at a glance, that something has broken, where it is, how bad it is, and what to do next.

The quality of this interface — what you chose to show, what you chose to leave out, and why — is a substantial part of what we are evaluating. You will be asked to explain your reasoning in writing.

6. A fault simulator

We cannot plug into a real substation. Ship a way for us to inject a fault into your system and watch it get detected, localized, ticketed, and closed. This is how we will actually evaluate your work, so make it easy to drive.

Where the AI part comes in

The role is AI Product Engineer, so we are looking at two things.

How you build. We expect you to use AI tooling heavily and we want to see how. You will document which tools you used, what you delegated, where the AI was wrong or misleading, and what you threw away. There is no penalty for AI having written most of the code. There is a penalty for not understanding it — we will ask.

Where AI belongs in the product. Somewhere in this system there is probably a place where an LLM earns its keep, and there are definitely places where it does not. Pick one AI-shaped feature, build it, and justify it in a paragraph. If you conclude that no part of this product should use an LLM, that is a legitimate answer — argue it and we will read the argument on its merits.

Be warned: reaching for an LLM to do the fault localization itself is a choice we will interrogate hard.

What we are not asking for

- Crew routing, vehicle allocation, or scheduling optimization
- Real authentication, SSO, or role-based permissions (a stub is fine)
- A mobile app
- Any actual hardware or firmware
- Historical analytics, reporting, or predictive maintenance
- Handling more than one city division

Building any of these instead of the core is a scoping failure, not bonus credit.

Constraints

- **Stack is yours to choose.** Use what you are fastest in. We have no preference and no hidden favourite.
- **Everything must run from a clean clone with one command.** Dockerized, `docker compose up`, seeded, working.

- **Everything must also be live on a public URL** we can open without installing anything or creating an account.
 - **Assume we cannot ask you questions.** We review submissions by email, in batch, with no back-and-forth. Anything you would have explained on a call has to be in your documentation.
-

Getting unstuck

The brief is deliberately incomplete in places. Where it is ambiguous, make a decision, write down the assumption, and move on. Documented assumptions are treated as correct answers even when we would have chosen differently.

If something is genuinely blocking — a broken link, a contradiction between two documents — email **[HIRING CONTACT EMAIL]** and we will answer factual questions. We will not answer "is this approach right?"

Full deliverable checklist and submission instructions: [03-deliverables-and-submission.md](#) (~~03-deliverables-and-submission.md~~).